

Today's Agenda

Artificial NN

Perceptron

Multi-layer FNN

Supervised learning

Classification problem

Q&A and Coding

Feedforward neural networks

What is Artificial Neural Network?

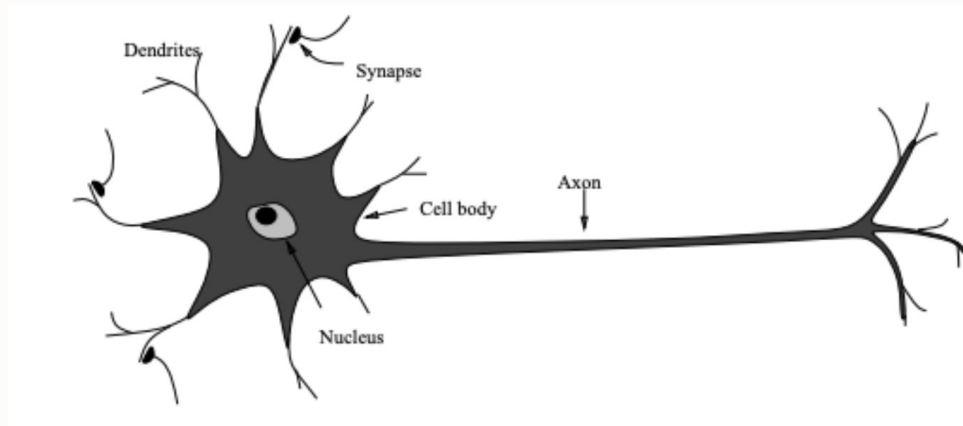
Artificial Neural Network

- ML discipline roughly is inspired by how neurons in a human brain work
- There are various types of neural networks (Feedforward, Recurrent, Convolutional)
- FNN applied to classification, clustering, regression, and association



Inspiration for neural networks

- Neuron is a special biological cell with information processing ability
 - It receives signals from other neurons through its *dendrites*
 - If the signals received exceeds a threshold, the neuron fires
 - Neuron transmits signals to other neurons via its *axon*
- *Synapse*: contact between axon of a neuron and dendrite of another neuron
 - Synapse enhances or inhibits the signal that passes through it
 - Learning occurs by changing the effectiveness of synapse



Cerebral cortex

- Outermost layer of brain, 2 to 3 mm thick, surface area of 2,200 sq. cm
- Has about 10^{11} neurons
 - Each neuron connected to 10^3 to 10^4 neurons
 - Human brain has 10^{14} to 10^{15} connections

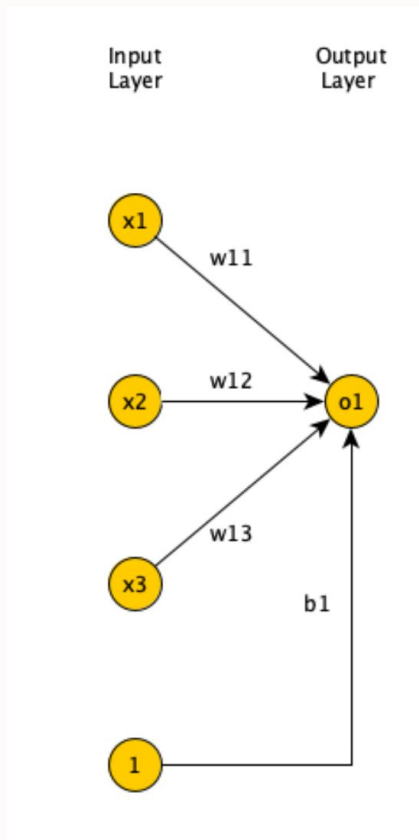


Cerebral cortex

- **Neurons communicate using signals that last for milliseconds**
 - Signal transmission frequency can reach several hundred Hertz
 - Millions of times slower than an electronic circuit
 - Complex tasks like face recognition are completed within a few hundred ms
 - The computation involved cannot take more than 100 serial steps
- **The information sent from one neuron to another is very small**
 - Critical information is not transmitted directly
 - It is captured by the interconnections
- **Distributed computation/representation of the brain**
 - Allows slow computing elements to perform complex tasks quickly

Perceptron

Perceptron



Learning in Perceptron

- Given a set of input-output pairs (called *training set*)
- Learning algorithm iteratively adjusts model parameters
 - Weights and biases
- So the model can accurately map inputs to outputs
- Perceptron learning algorithm

Limitations of Perceptron

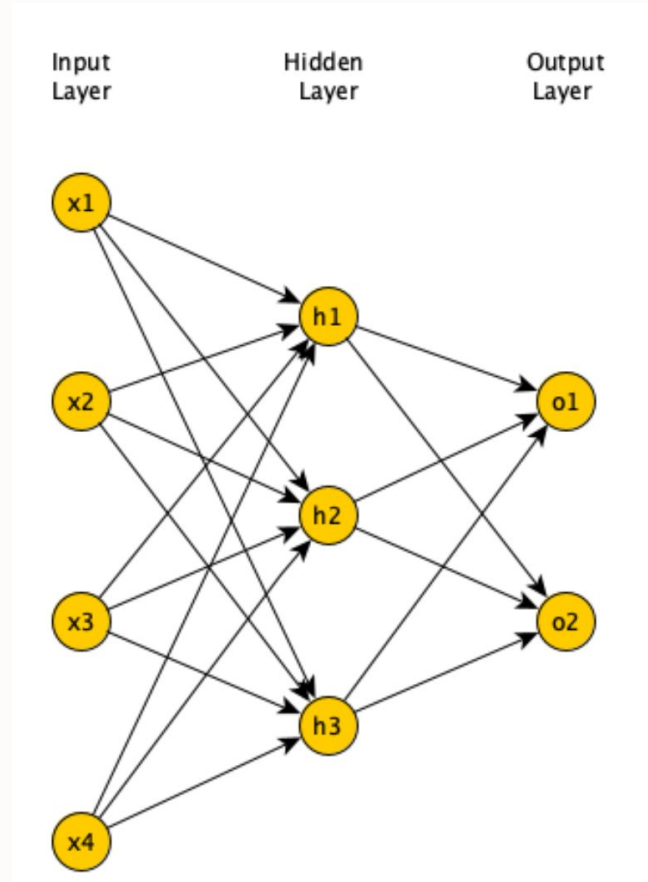
- Single layer FNN cannot solve problems in which data is not linearly separable
 - E.g., the XOR problem
- Adding one (or more) hidden layers enables FNN to represent any function
 - *Universal Approximation Theorem*
- Perceptron learning algorithm could not extend to multi-layer FNN
 - AI winter
- *Backpropagation* algorithm in 80's enabled learning in multi-layer FNN

Multi-layer FNN

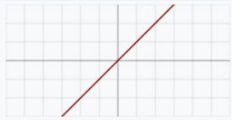
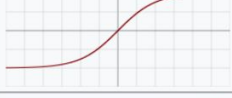
Multi-layer FNN

More hidden layers (and more neurons in each hidden layer)

- Can estimate more complex functions
- More parameters increase training time
- The likelihood of *overfitting* is higher

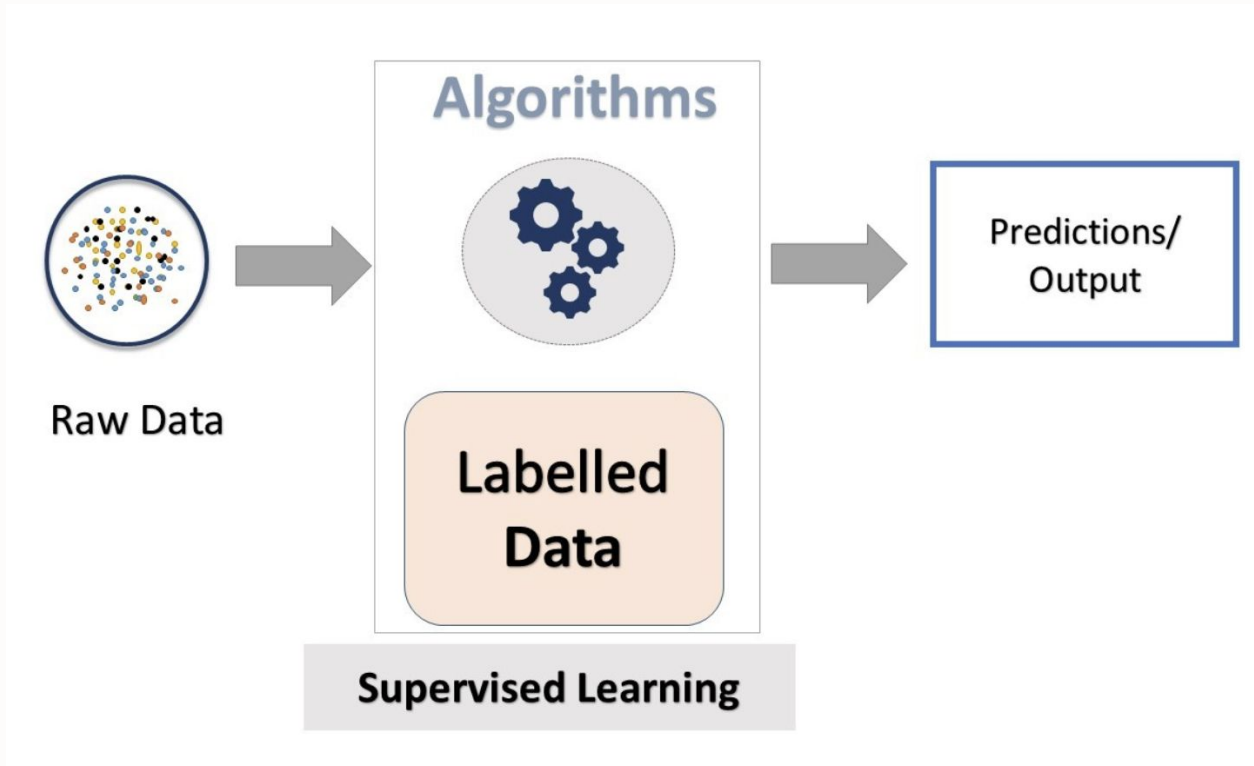


Activation functions

Name	Plot	Function, $f(x)$	Derivative of f , $f'(x)$	Range
Identity		x	1	$(-\infty, \infty)$
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$\begin{cases} 0 & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	$\{0, 1\}$
Logistic, sigmoid, or soft step		$\sigma(x) = \frac{1}{1 + e^{-x}}$ ^[1]	$f(x)(1 - f(x))$	$(0, 1)$
tanh		$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - f(x)^2$	$(-1, 1)$
Rectified linear unit (ReLU) ^[11]		$\begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ $= \max\{0, x\} = x \mathbf{1}_{x>0}$	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	$[0, \infty)$

Supervised learning

Supervised learning



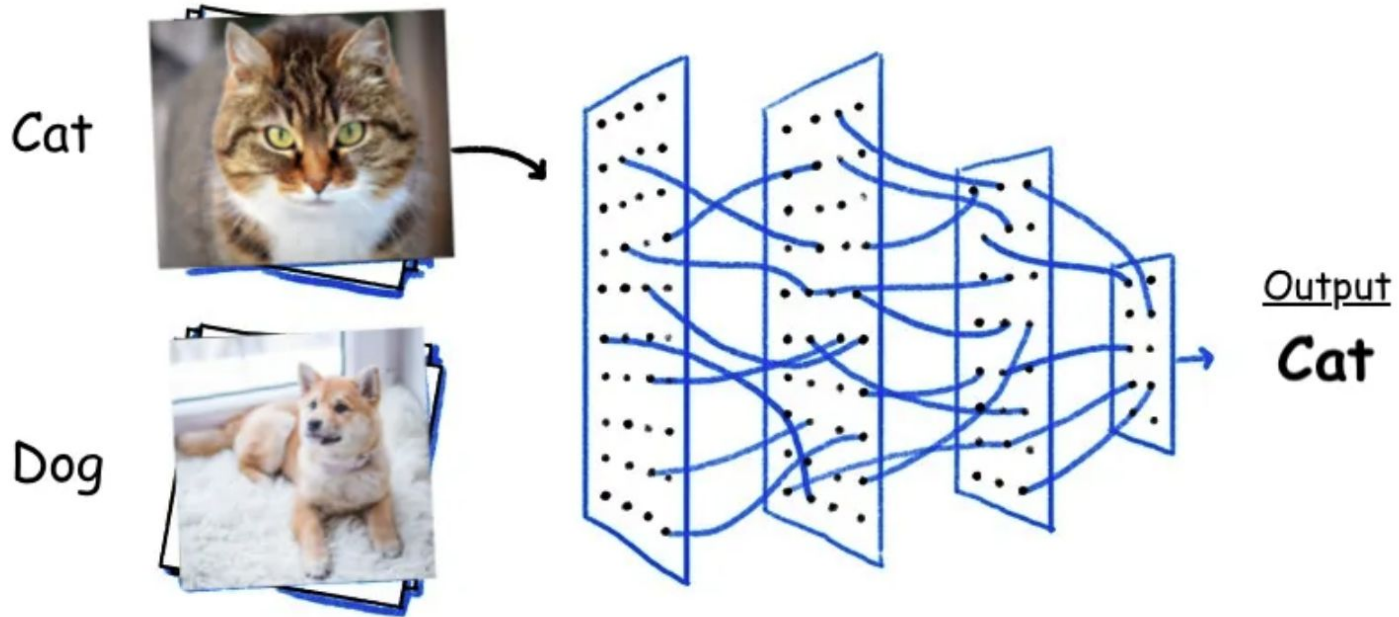
Supervised learning

- Training set of size m : $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$
- x_i is called feature vector
- Each element of feature vector is called a feature
- Each x_i corresponds to a label y_i

Supervised learning

- We assume an unknown function $y = f(x)$ that maps feature vectors to labels
- The goal is to use the training set to learn or estimate f
 - We want the estimate to be close to $f(x)$ not only for training set
 - But for testing (unseen) samples

Supervised learning



Classification problem

Classification problem

Patient data

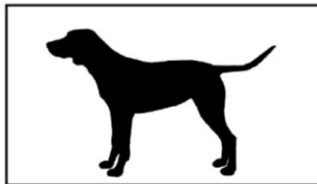
Binary
classification

$\mathbf{x}^{(i)} =$

$$\mathbf{y}^{(i)} = \begin{cases} 1 & \text{Disease} \\ 0 & \text{No disease} \end{cases}$$

Multiclass
classification

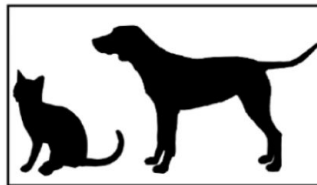
$\mathbf{x}^{(i)} =$



$$\mathbf{y}^{(i)} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{matrix} \text{Dog} \\ \text{Panda} \\ \text{Cat} \end{matrix}$$

Multilabel
classification

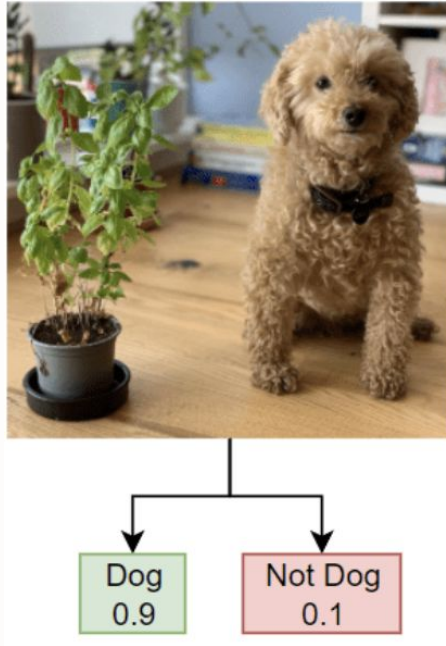
$\mathbf{x}^{(i)} =$



$$\mathbf{y}^{(i)} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \begin{matrix} \text{Dog} \\ \text{Panda} \\ \text{Cat} \end{matrix}$$

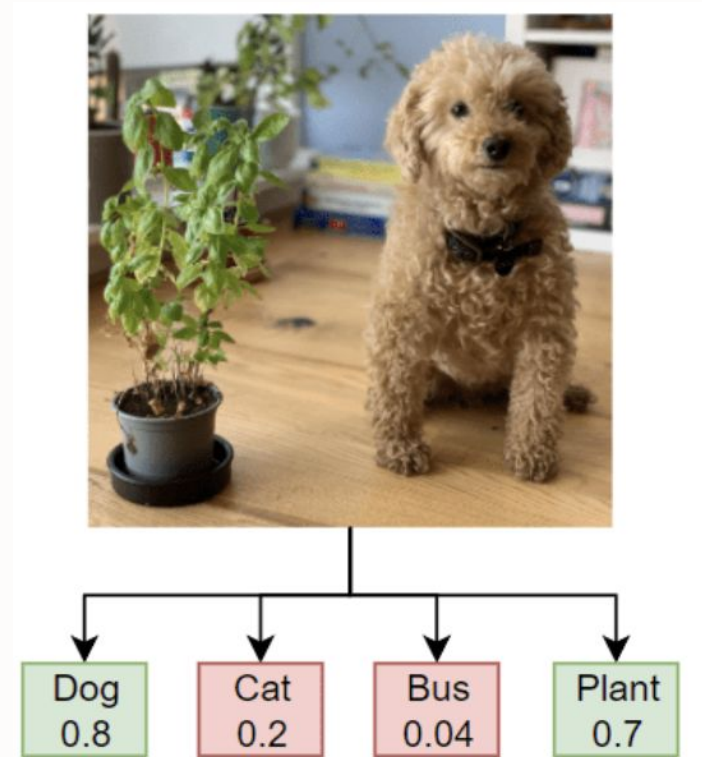
Output layer - Binary classification

- Single neuron in output layer
- Sigmoid activation function
- Activation > 0.5 , output 1
- Activation ≤ 0.5 , output 0



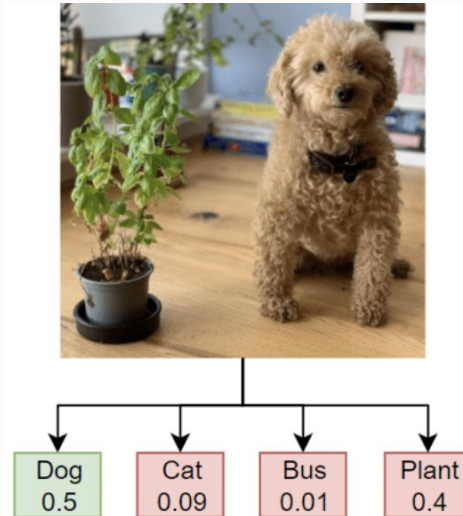
Output layer - Multilabel classification

- As many neurons in output layer as number of classes
- Sigmoid activation function
- Activation > 0.5 , output 1
- Activation ≤ 0.5 , output 0



Output layer - Multiclass classification

- As many neurons in output layer as number of classes
- *Softmax* activation function
- Takes input to neurons in output layer
- Creates a probability distribution, sum of outputs adds up to 1
- The neuron with the highest probability is the predicted label



Loss/Cost functions

Loss / Cost functions

- During training each sample is fed into the neural network
- We compare predicted output with true label
- The loss function measures the difference between predicted and expected output

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^c y_j \log \hat{y}_j$$

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i)$$

Cross-Entropy Loss for Classification

Loss for a Single Example

$$\mathcal{L}(\hat{y}, y) = - \sum_{j=1}^c y_j \log \hat{y}_j$$

- $\hat{y}_j$: predicted probability for class j
- y_j : true label (one-hot encoded, so only one $y_j = 1$)
- c : total number of classes
- This is the **cross-entropy loss** — it penalizes the model when it assigns low probability to the true class.

Cross-Entropy Loss for Classification

Cost Function over Dataset

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i)$$

- m : number of training examples
- $\hat{y}_i, y_i$: prediction and label for the i -th example
- $\mathbf{W}, \mathbf{b}$: model parameters
- This is the **average loss** over the whole dataset — used to train the model.

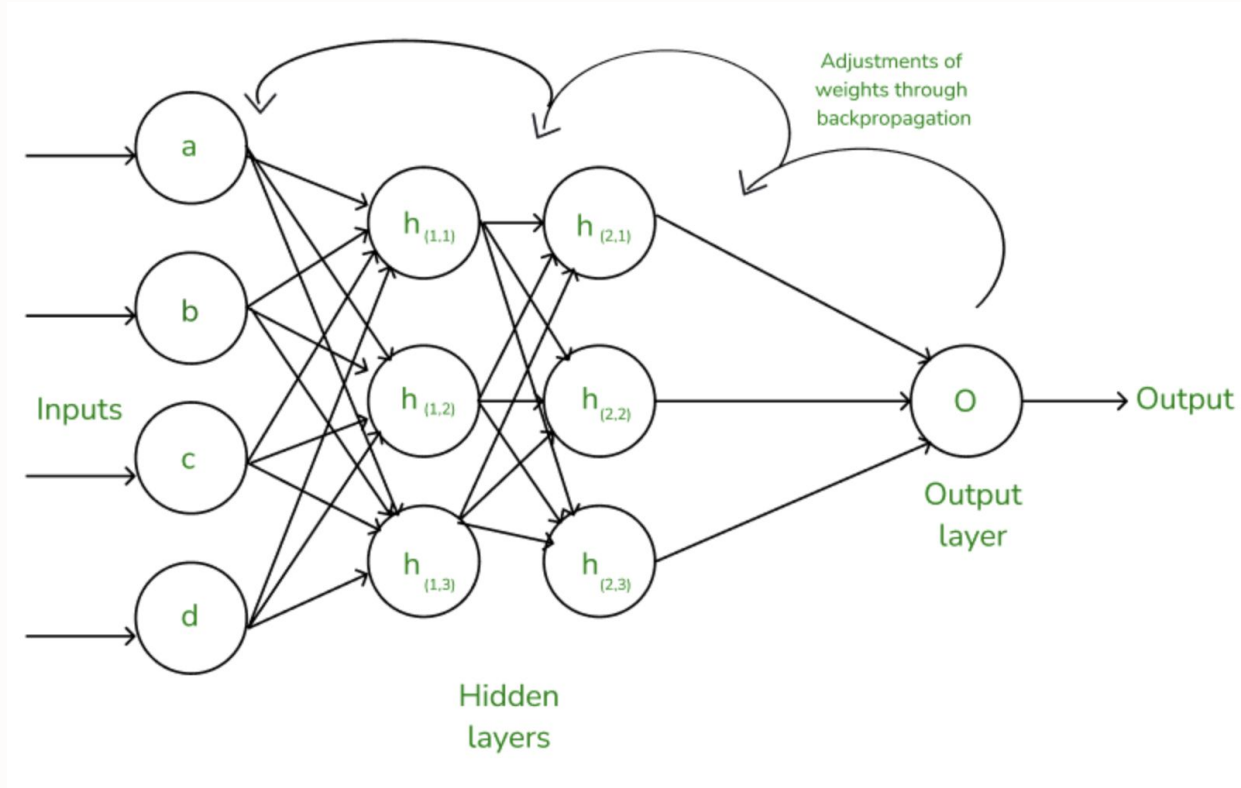
Backpropagation learning algorithm

Backpropagation learning algorithm

- The cost function depends on the weights and biases of all neurons across all layers
- Backpropagation iteratively computes the gradient of the cost function with respect to each weight and bias
- Weights and biases are updated in the opposite direction of the gradient
- Gradients (partial derivatives) guide these updates to minimize the cost function and reach a local minimum

Gradient descent technique: Finds a local minimum of a function by iteratively moving in the direction opposite to the gradient at the current point.

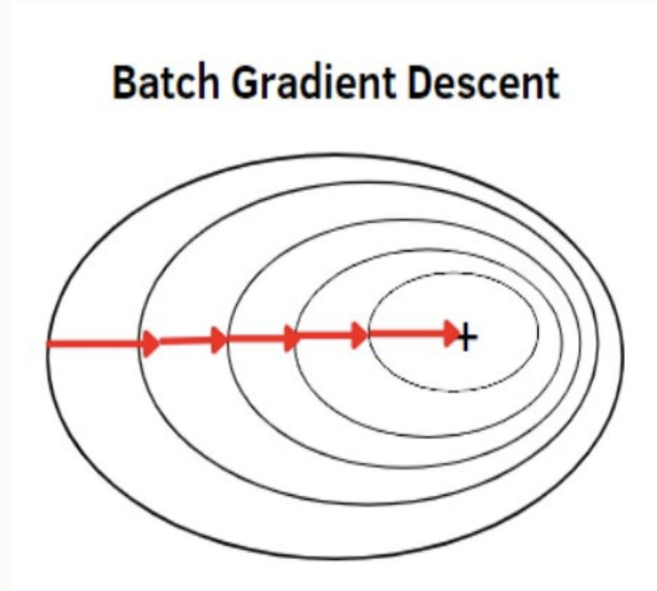
Backpropagation learning algorithm



Types of gradient descent

Batch gradient descent

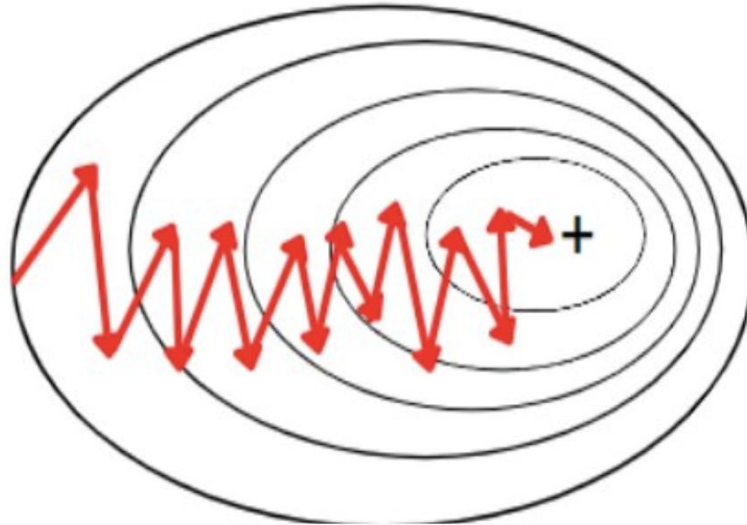
- Calculates gradient for each weight/bias for *all* samples
- Averages gradients and updates weights/biases
- Slow, if we have too many samples



Stochastic gradient descent

- Updates weights/biases based on the gradient of *each* sample
- Fast. Not accurate if sample gradient is not representative

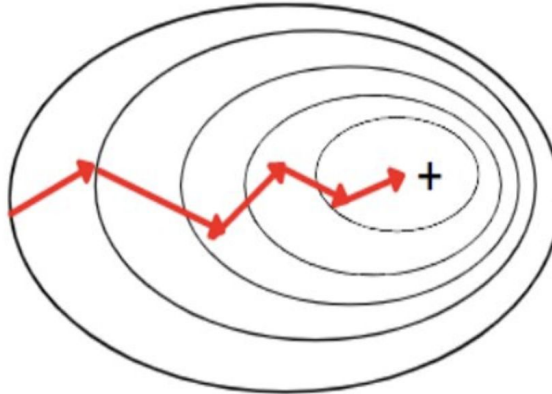
Stochastic Gradient Descent



Mini-batch gradient descent

- Middle ground solution
- Calculates gradient for each weight/bias for all samples in a *batch*
 - batch size is much smaller than training set size
- Averages batch gradients and updates weights/biases

Mini-Batch Gradient Descent



<Coding time>

Q&A time!